VISVESVARAYA TECHNOLOGICAL UNIVERSITY

Belagavi, Karnataka

MINI PROJECT REPORT

Smart Pharmacy Inventory Management System

Submitted in partial fulfillment of the requirements for the award of

Bachelor of Engineering in Computer Science and Engineering

Submitted by

Name of Student 1

USN: 1XX21CS001

Name of Student 2

USN: 1XX21CS002

Under the guidance of

Guide Name

Designation, Department of CSE

Name of the College

Department of Computer Science and Engineering

Academic Year: 2025–2026

Certificate

CERTIFICATE

This is to certify that the mini project report entitled **Smart Pharmacy Inventory Management System** is a bonafide record of the work carried out by **Name of Student 1** and **Name of Student 2** in partial fulfillment of the requirements for the award of the degree of Bachelor of Engineering in Computer Science and Engineering of Visvesvaraya Technological University, Belagavi, during the academic year 2025–2026.

The project work has been carried out under the supervision of **Guide Name**, Designation, Department of CSE. The report submitted herewith is an original work and has not been submitted earlier to any other university or institution for the award of any degree or diploma.

Signature of Guide	**Signature of HOD**
Guide Name	HOD Name
Designation	Head of Department
Department of CSE	Department of CSE
Signature of Principal	**External Examiner**
Principal Name	Name
Principal	Examiner

Declaration

We hereby declare that the mini project report entitled **Smart Pharmacy Inventory Management System** is the result of our own work carried out under the guidance of **Guide Name**. We also declare that this report has not been submitted previously, in whole or in part, to any university or institution for the award of any degree or diploma.

We further declare that all sources of information used in the preparation of this report have been duly acknowledged and referenced. Any resemblance to existing work is purely coincidental unless explicitly cited in the references section.

Place: _____ Signature of Student 1: _____

Date: _____ Signature of Student 2: _____

Acknowledgement

We express our sincere gratitude to the management of the institution for providing the necessary facilities and academic environment for the successful completion of this mini project. We are deeply thankful to the Head of the Department, faculty members, and lab staff of the Department of Computer Science and Engineering for their continuous support, encouragement, and valuable guidance.

Our heartfelt thanks go to **Guide Name**, whose suggestions, technical insights, and patient supervision were instrumental throughout the development of the system and the preparation of this report. We also thank our parents and friends for their motivation and support during the project period.

The completion of this project has been a meaningful learning experience in web development, database design, software testing, and documentation. The knowledge gained during this work will remain valuable for future academic and professional endeavors.

Abstract

The **Smart Pharmacy Inventory Management System** is a web-based solution developed to streamline pharmacy operations such as medicine inventory control, billing, customer ordering, delivery tracking, and report generation. Traditional pharmacy workflows often rely on manual registers or loosely integrated tools, which can lead to stock mismanagement, billing delays, duplicate entries, poor traceability, and inefficiencies in customer service. This project addresses those issues by providing a unified platform that combines administrative, operational, and customer-facing functions in a single application.

The system is implemented using **HTML, CSS, JavaScript, Bootstrap, PHP, MySQL, and XAMPP**. It supports multi-role authentication for admin, employees, and customers, each with a dedicated dashboard and access control mechanism. The inventory module enables medicine management with expiry and low-stock alerts, while the billing module supports invoice generation and email delivery. The customer module allows online browsing, shopping cart management, checkout, and order tracking. Delivery and sales records are maintained in a structured database to support analysis and reporting.

The proposed system improves operational accuracy, reduces manual workload, and enhances customer satisfaction through better data visibility and faster processing. It also provides management with actionable insights through analytical reports on sales, stock movement, and order status. Overall, the project demonstrates a practical and scalable approach to digital pharmacy management for academic and real-world use.

Contents

Certificate i

Declaration ii

Acknowledgement iii

Abstract iv

1 Introduction 1

1.1 Introduction . . . 1
1.2 Motivation . . . 2
1.3 Problem Statement . . . 2
1.4 Objectives . . . 3
1.5 Scope of Project . . . 3
1.6 Existing System . . . 4
1.7 Proposed System . . . 4
1.8 Advantages . . . 5
1.9 Organization of Report . . . 5

2 System Requirements Specification 7

2.1 Hardware Requirements . . . 7
2.2 Software Requirements . . . 7
2.3 Functional Requirements . . . 8
2.4 Non-Functional Requirements . . . 9
2.5 Feasibility Study . . . 10
2.5.1 Technical Feasibility . . . 10
2.5.2 Economic Feasibility . . . 10
2.5.3 Operational Feasibility . . . 10

2.5.4 Schedule Feasibility 11

3 System Design **12**

3.1 System Architecture 12

3.2 Use Case Diagram 13

3.3 ER Diagram 13

3.4 Data Flow Diagram 14

3.5 Database Design 15

3.6 Module Description 16

3.6.1 Admin Module 16

3.6.2 Employee Module 16

3.6.3 Billing Module 17

3.6.4 Customer Module 17

3.6.5 Inventory Module 17

3.6.6 Delivery Tracking Module 18

3.6.7 Reports Module 18

4 Implementation **19**

4.1 Frontend Implementation 19

4.2 Backend Implementation 19

4.3 Database Connectivity 20

4.4 Login Authentication 20

4.5 Billing Flow 21

4.6 Inventory Management 21

4.7 Customer Ordering 22

4.8 Delivery Management 22

4.9 Reports Generation 22

5 Testing **24**

5.1 Unit Testing 24

5.2 Integration Testing 24

5.3 System Testing 25

5.4 Validation Testing 25

5.5 Test Cases 25

5.6 Result Analysis 26

6 Results and Discussion **27**

6.1 Screenshot Placeholders 28

7 Conclusion and Future Enhancements **33**

7.1 Conclusion 33

7.2 Future Enhancements 33

References **35**

List of Figures

3.1 System Architecture of Smart Pharmacy Inventory Management System 12
3.2 Use Case Diagram 13
3.3 ER Diagram 14
3.4 Data Flow Diagram 14

6.1 Login Page 28
6.2 Registration Page 28
6.3 Admin Dashboard 28
6.4 Billing Page 29
6.5 Invoice Generation 29
6.6 Medicines Page 29
6.7 Reports Page 30
6.8 Delivery Tracking 30
6.9 Customer Dashboard 30
6.10 Shop Page 31
6.11 Cart Page 31
6.12 Checkout Page 31
6.13 My Orders Page 32

List of Tables

2.1 Hardware Requirements 7
2.2 Software Requirements 8
2.3 Functional Requirements 9

3.1 Database Tables and Their Purpose 15

5.1 Sample Test Cases 25

List of Snapshots

Login Page 28
Registration Page 28
Admin Dashboard 28
Billing Page 29
Invoice Generation 29
Medicines Page 29
Reports Page 30
Delivery Tracking 30
Customer Dashboard 30
Shop Page 31
Cart Page 31
Checkout Page 31
My Orders Page 32

Chapter 1

Introduction

1.1 Introduction

The pharmacy sector requires accurate inventory control, timely billing, dependable customer service, and traceable order handling. A single error in stock entry or invoice processing can disrupt the availability of essential medicines and affect patient care. In many small and medium-sized pharmacies, the operational workflow still depends on manual registers, spreadsheet files, or isolated software tools that do not communicate with one another. This creates problems such as duplicated records, delayed replenishment, improper tracking of expiry dates, and difficulty in producing management reports.

The **Smart Pharmacy Inventory Management System** is designed as an integrated web application to bring all pharmacy operations into one platform. The system connects stock management, billing, customer ordering, delivery monitoring, and reporting in a coordinated environment. It is built around role-based access so that administrators, employees, and customers can perform only the actions relevant to their responsibilities. This improves security, reduces confusion, and increases transparency across the workflow.

The project uses a standard web architecture with a PHP backend and MySQL database, deployed locally using XAMPP. The user interface is built using HTML, CSS, JavaScript, and Bootstrap to provide a responsive and practical experience on desktop systems. This makes the project suitable as a mini project for academic evaluation while also reflecting a realistic pharmacy automation scenario.

1.2 Motivation

The motivation for this project arose from the observation that pharmacies face a constant need to monitor medicine quantities, expiry dates, customer orders, billing records, and delivery status with high reliability. When these tasks are handled manually, the staff must spend significant time entering data, searching for records, and verifying availability. Such effort not only reduces productivity but also increases the possibility of human error.

Another important motivation is the growing expectation for digital services. Customers increasingly prefer to check product availability, place orders, and track deliveries online. At the same time, pharmacy managers need better visibility into sales trends, stock depletion, and supplier relationships. A software system that supports both internal operations and customer interactions provides measurable value by saving time, reducing wastage, and improving service quality.

The project was therefore developed to simulate a practical digital pharmacy environment that demonstrates full-stack web development, database design, access control, and report generation. It also serves as a learning platform for understanding how business requirements are translated into modules, tables, flows, and user interfaces.

1.3 Problem Statement

Pharmacies often experience operational challenges caused by disconnected workflows and limited automation. Common issues include inaccurate stock records, delays in manual billing, absence of low-stock alerts, poor visibility into medicine expiry, and lack of a reliable order-tracking mechanism. When the data is stored across multiple files or registers, maintaining consistency becomes difficult and reporting becomes slow.

The problem addressed by this project is the absence of a unified, role-based, and database-driven system that can manage inventory, billing, customer orders, delivery progress, and analytics in one place. Without such a system, pharmacy staff may spend more time correcting errors than serving customers. The lack of integration can also reduce accountability and make it hard for management to make informed decisions.

This project aims to solve those issues by designing a centralized application that automates

essential pharmacy operations, ensures traceability, and delivers timely information to users based on their role.

1.4 Objectives

The main objective of the project is to design and implement a smart pharmacy inventory management platform that simplifies daily operational tasks and improves data accuracy. The specific objectives are:

- To provide a unified multi-role login system for administrators, employees, and customers.
- To maintain medicine stock information with support for additions, updates, and deletions.
- To generate invoices automatically during billing and checkout.
- To enable customers to browse medicines, add them to a cart, and place orders online.
- To track delivery progress from dispatch to completion.
- To generate analytical reports for sales, stock levels, and orders.
- To send invoice details by email for record keeping and convenience.
- To display low-stock and expiry alerts for preventive action.

1.5 Scope of Project

The scope of the project covers the development of a complete pharmacy management workflow within a web application. It includes administration of medicines, employee interactions, customer ordering, billing, delivery monitoring, and reporting. The system is intended to support both operational and informational functions.

Within the administrative scope, the system allows the management of medicines, suppliers, users, employees, customers, and sales statistics. Within the customer scope, it allows product

browsing, cart management, checkout, order tracking, and order history review. Within the employee scope, it supports billing assistance, order preparation, and delivery updates.

The project is scoped as a mini project and therefore focuses on practical implementation of core modules rather than advanced enterprise features such as third-party payment gateway integration, barcode scanning hardware, or mobile app synchronization. Nevertheless, the design is modular and can be extended in the future.

1.6 Existing System

In a typical existing pharmacy setup, records are maintained manually or using basic standalone software. Stock registers are updated by hand, bills are issued using simple billing applications, and customer orders are often received by phone or in person. These methods work to a limited extent but create considerable inefficiency as business volume increases.

Manual systems are usually not capable of automatic low-stock alerts, expiry monitoring, or consolidated reporting. Searching for specific invoices or order records becomes time-consuming. Errors in quantity calculation or duplicate record entry can lead to financial loss and customer dissatisfaction. Since information is distributed across notebooks, spreadsheets, or disconnected files, the staff cannot obtain a unified view of pharmacy operations.

Although some partial digital solutions exist, they often focus on only one part of the workflow, such as billing or inventory alone. Such fragmented systems do not provide end-to-end traceability from stock entry to customer delivery. This limitation motivated the development of a more integrated system.

1.7 Proposed System

The proposed system is a web-based application with centralized data storage and role-based access control. All important activities are recorded in the MySQL database and accessed through secure PHP pages. The system provides a dashboard-based interface that presents relevant information to each user role.

The admin role manages master data and business controls. Employees work with billing, order handling, and operational updates. Customers can register, log in, browse medicines,

add products to the cart, place orders, and view order status. The system automatically tracks stock changes, alerts the user when a medicine reaches a low quantity threshold, and identifies medicines close to expiry.

The proposed approach improves speed, consistency, and accountability. It also facilitates better report generation because every transaction is stored in a structured format and can be summarized using SQL queries. The system is therefore more reliable than manual methods and more complete than fragmented point solutions.

1.8 Advantages

The proposed system offers several practical advantages:

- Centralized and consistent data storage.
- Faster billing and invoice generation.
- Improved visibility of stock levels and expiry dates.
- Easy management of customer orders and delivery tracking.
- Better reporting for decision-making and auditing.
- Reduced manual workload and fewer data entry errors.
- Secure role-based access to sensitive information.
- Email-based invoice delivery for digital records.

1.9 Organization of Report

This report is organized into seven chapters. Chapter 1 presents the introduction, motivation, problem statement, objectives, scope, and comparison between the existing and proposed systems. Chapter 2 covers system requirements specification, including hardware, software, functional, and non-functional requirements, along with feasibility analysis. Chapter 3 describes system design, architecture, diagrams, database design, and module descriptions. Chapter 4 explains implementation details covering frontend, backend, connectivity, authentication, billing, inventory, ordering, delivery, and reports.

Chapter 5 discusses testing strategies and representative test cases. Chapter 6 presents results and discussion with screenshot placeholders for the major pages of the application. Chapter 7 concludes the report and outlines future enhancements. The references section lists the sources and technologies that supported the project development.

Chapter 2

System Requirements Specification

2.1 Hardware Requirements

The system has been designed to run on a standard desktop or laptop computer with modest hardware resources. Since the application is implemented as a web-based local system using XAMPP, it does not require high-end infrastructure for development or demonstration.

Table 2.1: Hardware Requirements

Component	Requirement
Processor	Intel Core i3 or above, or equivalent AMD processor
RAM	4 GB minimum, 8 GB recommended
Storage	20 GB free disk space or more
Display	1366 x 768 resolution or higher
Input Devices	Keyboard, mouse, and optional printer for invoice output
Network	Optional LAN or internet connection for email delivery and deployment testing

2.2 Software Requirements

The software stack used in the project is chosen for simplicity, educational relevance, and compatibility with common web hosting environments.

Table 2.2: Software Requirements

Software	Requirement
Operating System	Windows 10/11 or any modern system supporting XAMPP
Web Server	Apache through XAMPP
Backend Language	PHP 7.4 or above
Database	MySQL/MariaDB
Frontend Technologies	HTML, CSS, JavaScript, Bootstrap
Editor	VS Code, Sublime Text, or similar IDE
Email Library	PHPMailer for invoice delivery
Browser	Chrome, Edge, Firefox, or equivalent

2.3 Functional Requirements

Functional requirements describe what the system must do. These requirements were derived from the expected workflow of a pharmacy and the intended features of the project.

Table 2.3: Functional Requirements

Requirement	Description
User Authentication	The system shall authenticate admins, employees, and customers using secure login credentials.
Medicine Management	The system shall allow insertion, updating, viewing, and deletion of medicine records.
Inventory Alerts	The system shall identify low-stock medicines and medicines nearing expiry.
Billing	The system shall generate bills and invoices automatically based on purchase items.
Shopping Cart	The system shall permit customers to add, update, or remove medicines before checkout.
Order Tracking	The system shall track order status from placement to delivery completion.
Reporting	The system shall generate summaries of sales, stock movement, and customer orders.
Email Invoice	The system shall send invoice information to the customer email address.

2.4 Non-Functional Requirements

Non-functional requirements describe how the system should behave in terms of reliability, performance, usability, maintainability, and security.

- **Performance:** The application should respond quickly for standard operations such as login, search, and bill generation.
- **Usability:** The interface should be simple and understandable for different user categories.
- **Security:** Access to data should be restricted based on user role and session control.
- **Reliability:** Data should be stored consistently and should remain available for future reference.

- **Maintainability:** The codebase should be modular so that future modifications are manageable.
- **Scalability:** The database design should support additional modules or data fields later.
- **Portability:** The application should run on common web servers and browsers without major modification.

2.5 Feasibility Study

Feasibility analysis helps determine whether the proposed system is practical from technical, economic, and operational perspectives.

2.5.1 Technical Feasibility

The project is technically feasible because the chosen technologies are mature, stable, and widely supported. PHP and MySQL are well-suited for lightweight business applications, and XAMPP simplifies local deployment for development and demonstration. Bootstrap assists in building responsive interfaces without high implementation complexity.

2.5.2 Economic Feasibility

The economic cost of the project is low because all major software components are open source or freely available for educational use. The system can be developed and tested on a standard personal computer without requiring expensive infrastructure. This makes the project affordable for academic purposes and attractive for small pharmacy deployments.

2.5.3 Operational Feasibility

The proposed system is operationally feasible because its workflow aligns with common pharmacy procedures. Staff members can quickly understand the role-based dashboard, and customers can use the shopping and tracking features with minimal training. The design reduces repetitive manual effort while preserving familiar business processes.

2.5.4 Schedule Feasibility

The project scope is suitable for a mini project timeline because the modules can be developed incrementally. Core database tables, authentication, billing, inventory, and reports can be completed in phases. This makes the project manageable within an academic semester.

Chapter 3

System Design

3.1 System Architecture

The system follows a three-tier web architecture consisting of presentation, application, and data layers. The presentation layer contains the HTML, CSS, JavaScript, and Bootstrap interfaces accessed by users through a browser. The application layer contains PHP scripts that process requests, apply business logic, and manage user sessions. The data layer consists of the MySQL database that stores medicine, customer, invoice, order, delivery, and report information.

This separation of concerns improves maintainability and supports cleaner code organization. It also makes it easier to debug modules independently because each tier has a clearly defined responsibility. The architecture is suitable for both local demonstration and future hosting on a web server.

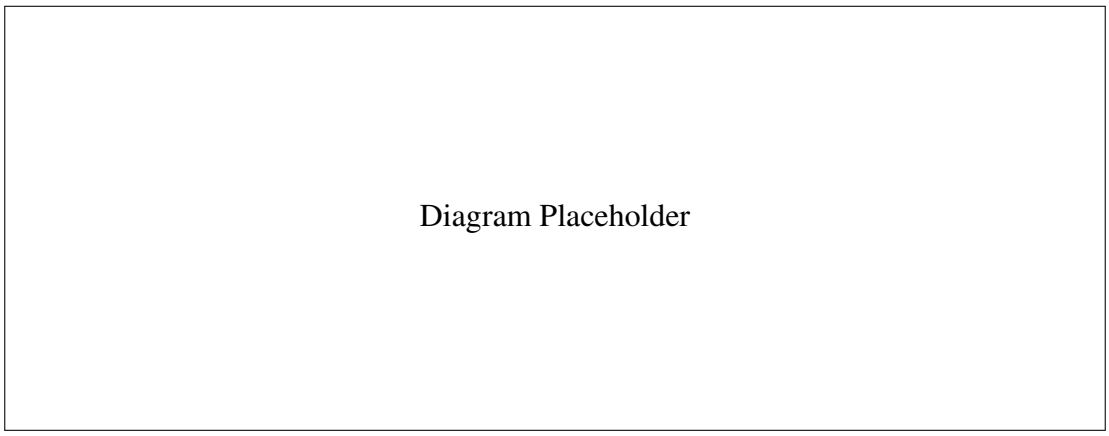


Figure 3.1: System Architecture of Smart Pharmacy Inventory Management System

3.2 Use Case Diagram

The use case view of the system shows how different actors interact with the application. The primary actors are Admin, Employee, and Customer. The admin manages master records and reports. The employee handles billing, order assistance, and operational updates. The customer registers, logs in, browses medicines, places orders, and tracks delivery.

The following use case diagram placeholder can be replaced with the final graphical version during submission.

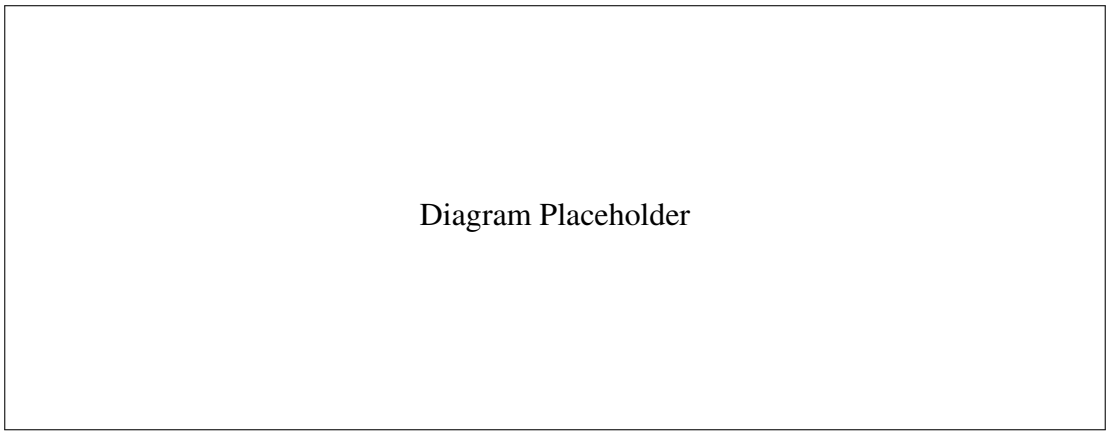


Figure 3.2: Use Case Diagram

3.3 ER Diagram

The Entity Relationship Diagram defines the logical relationships among the database tables. The design includes tables for user authentication, medicine catalog, cart, billing, customer profiles, employee records, delivery information, sales history, suppliers, and order details. The ER structure supports one-to-many relationships between customers and orders, orders and order items, and orders and delivery tracking records.

The central idea of the design is to avoid redundancy while preserving enough detail for traceability. For example, a single order is linked to multiple items through the user_order_items table, and each bill can be traced back to a customer and a sales transaction. Such relationships make reporting and auditing more reliable.

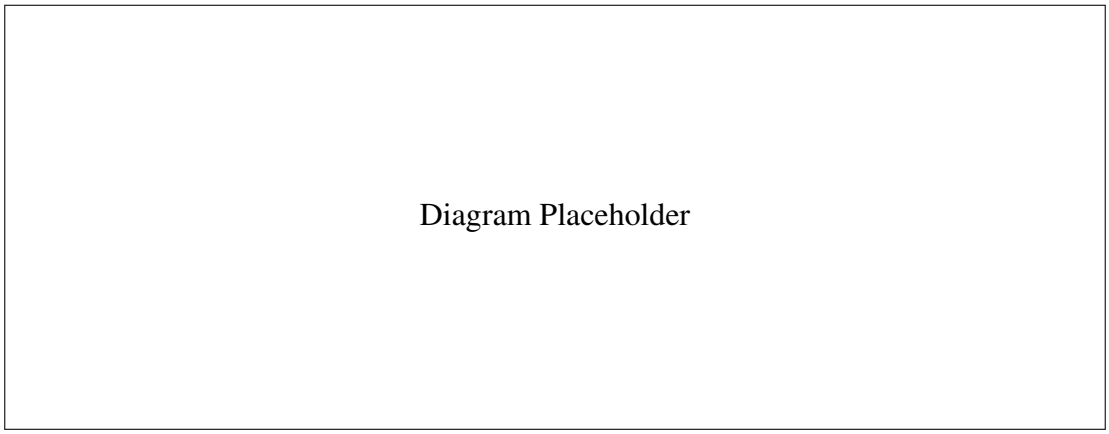


Figure 3.3: ER Diagram

3.4 Data Flow Diagram

The Data Flow Diagram represents how information moves through the system. At the highest level, the customer or employee enters data into the system through the web interface. The application validates the input, performs the required processing, and stores the information in the database. The resulting output may include a bill, updated stock level, order confirmation, or a report.

At the lower level, the system processes include login authentication, medicine management, billing, checkout, delivery update, and analytics. Each process communicates with the database and produces information that is either shown to the user or used by another module. The DFD helps in understanding the operational flow before implementation.

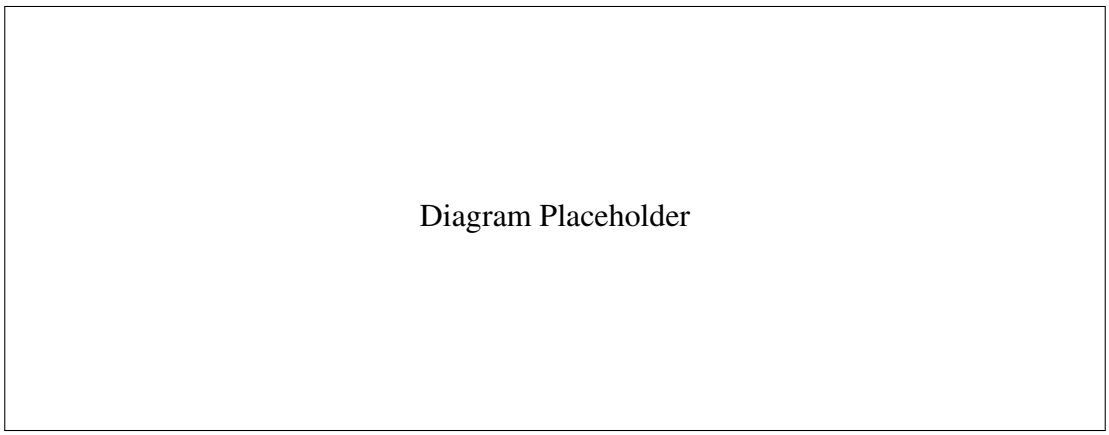


Figure 3.4: Data Flow Diagram

3.5 Database Design

The database design is based on relational modeling principles. Each table represents a distinct business entity or transaction. Primary keys uniquely identify rows, while foreign keys are used to connect related records. The database is structured to support data integrity, query efficiency, and future expansion.

Table 3.1: Database Tables and Their Purpose

Table Name	Purpose	Key Attributes
admin_users	Stores admin login credentials	admin_id, username, password, role
users	Stores customer login information	user_id, name, email, password, phone
medicine	Maintains medicine catalog and stock	medicine_id, name, batch, price, quantity, expiry_date
cart	Stores customer cart sessions	cart_id, user_id, created_at
cart_items	Stores items added to cart	cart_item_id, cart_id, medicine_id, quantity
bill	Stores generated invoices	bill_id, bill_date, total_amount, customer_id
customer	Stores customer profile data	customer_id, name, email, address, contact
delivery_orders	Stores delivery requests	delivery_id, order_id, status, assigned_agent
delivery_tracking	Stores tracking updates	tracking_id, delivery_id, location, status, updated_at
employee	Stores employee details	employee_id, name, designation, contact
employee_login	Stores employee credentials	login_id, employee_id, username, password

Table Name	Purpose	Key Attributes
`sales`	Stores sales transaction history	sales_id, bill_id, medicine_id, quantity, amount
`supplier`	Stores supplier information	supplier_id, supplier_name, contact, address
`user_orders`	Stores customer order headers	order_id, user_id, order_date, status
`user_order_items`	Stores order line items	order_item_id, order_id, medicine_id, quantity, price

3.6 Module Description

The application is divided into multiple functional modules so that each responsibility remains clearly defined and easier to maintain.

3.6.1 Admin Module

The admin module provides complete control over the application. It includes management of medicines, employees, suppliers, customers, orders, and reports. The admin can monitor the overall status of the pharmacy, inspect low-stock items, review expiry warnings, and access analytics for better decision-making. This module serves as the supervisory layer of the system.

Administrators also manage user credentials and ensure that only valid users are allowed to access protected resources. In an operational environment, this module would be used by the pharmacy owner or manager to maintain oversight of the business.

3.6.2 Employee Module

The employee module is intended for staff members who perform day-to-day tasks such as assisting with billing, updating order status, checking stock levels, and recording operational updates. Employees do not require full access to master data; instead, they are given a limited interface that helps them complete assigned tasks efficiently.

This separation reduces accidental modification of sensitive records and improves account-

ability. It also simplifies the employee workflow because the interface displays only the actions relevant to service and operations.

3.6.3 Billing Module

The billing module allows the system to calculate item totals, discounts if applicable, and the final payable amount. Once the bill is generated, the invoice can be stored in the database and optionally sent by email to the customer. The module also links with the sales table so that each transaction is preserved for accounting and reporting.

The billing process is a critical component because it connects stock reduction, financial records, and customer service. Fast and accurate billing improves the user experience and reduces queue time at the pharmacy counter.

3.6.4 Customer Module

The customer module handles registration, login, browsing, shopping cart operations, checkout, and order tracking. Customers can search for medicines, add products to the cart, review item quantities, and confirm purchase requests. After checkout, they can observe order status through the dashboard or the dedicated tracking interface.

By giving customers visibility into their own orders, the system improves trust and convenience. It also helps reduce manual communication between customers and staff because the status information is accessible online.

3.6.5 Inventory Module

The inventory module maintains medicine details such as name, batch number, price, quantity, supplier, and expiry date. It allows the admin to update stock entries when new medicines arrive or when quantities change. The module also identifies low-stock items and expiry-prone medicines so that action can be taken before shortages occur.

Inventory management is one of the most important parts of the system because it directly affects availability, revenue, and wastage. Accurate inventory data ensures that the pharmacy does not sell medicines that are expired or unavailable.

3.6.6 Delivery Tracking Module

The delivery tracking module manages the movement of customer orders after checkout. It stores dispatch and tracking updates such as confirmed, packed, out for delivery, and delivered. If a delivery agent is assigned, the system can map the order to the person responsible for delivery.

This module improves the end-to-end order lifecycle by making the delivery process visible to both staff and customers. It is especially useful for online pharmacy workflows where home delivery is an expected service.

3.6.7 Reports Module

The reports module provides management-level summaries that support analysis and planning. It can display medicine stock reports, sales summaries, order statuses, low-stock alerts, and expiry-related information. Reports are generated from the transactional tables in the database and can be tailored to a chosen date range.

The availability of reports transforms stored data into actionable knowledge. It helps management identify popular medicines, monitor sales trends, and make procurement decisions based on evidence rather than guesswork.

Chapter 4

Implementation

4.1 Frontend Implementation

The frontend of the project is built with HTML, CSS, JavaScript, and Bootstrap. HTML is used to structure pages such as login forms, dashboards, medicine listings, cart pages, and invoices. CSS provides the visual styling that makes the application readable and consistent. Bootstrap is used to maintain a responsive layout and to simplify form, button, table, and card styling.

JavaScript is used to improve interactivity on the client side. Typical tasks include validation of forms before submission, dynamic updates in the cart, and confirmation prompts for sensitive actions such as deleting records. The frontend has been designed to present a clean and functional interface rather than a purely decorative one.

The user interface follows a dashboard-driven style. Each role sees a separate menu and data panel, which reduces confusion and helps users focus on their actual work. This implementation approach is suitable for an academic project because it demonstrates both usability and modular design.

4.2 Backend Implementation

The backend is developed using PHP. PHP scripts handle form submissions, session management, database queries, and business logic. Each module of the system has corresponding PHP files that interact with the MySQL database using secure query patterns.

The backend controls the sequence of operations. For example, after a customer adds an item to the cart, the backend validates the stock, stores cart details, and later converts the cart into an order during checkout. Similarly, when a bill is created, the backend updates the sales record

and prepares invoice data for display or email transmission.

The use of PHP makes the implementation lightweight and easy to deploy under XAMPP. It also allows the project to remain compatible with widely supported hosting environments, which is useful if the system is later expanded or hosted online.

4.3 Database Connectivity

Database connectivity is established through PHP MySQL functions or the mysqli extension. The application connects to the database using configuration details stored in a common file so that the connection can be reused by multiple modules. This reduces redundancy and centralizes configuration management.

All data-related operations are performed through SQL queries. The database stores structured information about users, medicines, orders, carts, bills, employees, suppliers, and delivery logs. Because the application is transactional in nature, insert, update, and select queries are used frequently. Proper connection handling ensures that data remains synchronized across different modules.

In a practical deployment, the database credentials should be protected and the connection file should be included securely. For the purposes of the mini project, the design emphasizes clarity and functionality in a local development environment.

4.4 Login Authentication

The authentication module validates users before allowing access to role-specific dashboards. During login, the system checks the supplied username or email and password against the corresponding table. Based on the authenticated role, the user is redirected to the appropriate page.

Session variables are used to remember logged-in users across multiple pages. If a user is not authenticated, access to protected content is denied and the user is redirected to the login page. This mechanism is essential for security because pharmacy data may include sensitive business and customer information.

The unified login flow simplifies the user experience because different roles do not need

separate applications. Instead, the same application identifies the role and presents the correct interface after successful authentication.

4.5 Billing Flow

The billing flow begins when a customer or employee selects medicines for purchase. The system calculates item totals based on quantity and unit price. The bill is then generated with a unique identifier and date stamp. If the invoice is sent by email, the customer receives a digital copy for reference.

Once the bill is finalized, the transaction is recorded in the sales table. The inventory quantity is reduced accordingly so that stock levels remain accurate. This tight coupling between billing and stock management prevents inconsistencies and ensures that the medicine availability shown to users reflects real data.

The billing process also supports transparency during customer service. Since the final amount is computed from database values, the possibility of manual arithmetic mistakes is reduced. This is especially useful when multiple products are purchased in a single transaction.

4.6 Inventory Management

Inventory management is implemented around the medicine table, where each record holds product details, stock count, expiry date, and pricing information. The admin can add new medicine records when fresh stock arrives and can update existing records when quantities change. Deletions are allowed only when necessary and should be handled carefully to preserve historical records.

Low-stock alerts are generated by checking medicine quantities against a threshold value. Expiry alerts are generated by comparing expiry dates with the current date or a warning interval. These features help the pharmacy avoid stockouts and reduce wastage from expired items.

Search and filter functionality can be used to locate medicines by name, category, or supplier. This improves accessibility in the database and makes the system more practical for daily operations.

4.7 Customer Ordering

Customer ordering begins with browsing medicines on the shop page. The customer can view available products, place selected medicines into the cart, and review the order before checkout. The cart module stores temporary selections while the order module stores confirmed purchases.

During checkout, the system verifies the selected items, computes the amount, and records the order in the database. Once placed, the order appears in the customerś order history and can be tracked through the my orders page. This process makes the shopping experience convenient and traceable.

The ordering workflow is one of the most visible features of the system because it links product browsing, transaction creation, and delivery tracking in a user-friendly manner.

4.8 Delivery Management

Delivery management tracks the progress of medicine orders after they leave the pharmacy. Each order may be associated with a delivery record that stores the status, location updates, and responsible agent if applicable. The delivery dashboard presents these updates to the staff so that they can monitor outstanding tasks.

The delivery status typically moves through stages such as confirmed, packed, dispatched, out for delivery, and delivered. This structured status flow improves transparency and creates a clear audit trail for completed orders. It also reduces the need for repeated customer inquiries because the current state can be checked within the system.

4.9 Reports Generation

Reports are generated using SQL aggregation queries over the transactional database tables. Typical reports include daily sales, total revenue, low-stock medicines, expiring medicines, customer orders, and delivery status summaries. These reports are presented in readable tables and can also support future export functionality.

The purpose of reporting is not only record keeping but also decision support. For instance, if a particular medicine sells quickly, the admin can increase the reorder quantity. If a batch

approaches expiry, the staff can prioritize its clearance. Thus, reporting contributes directly to both efficiency and financial control.

Chapter 5

Testing

5.1 Unit Testing

Unit testing was performed on individual modules to confirm that they behave correctly in isolation. Login validation, medicine insertion, cart updates, invoice creation, and order status changes were tested separately. The objective of unit testing was to ensure that each function or page performed its intended task without depending on the behavior of unrelated modules.

In the context of this project, unit testing is especially important because the application is built from multiple PHP scripts. If one script fails, it can affect the user workflow. Testing each module individually helps identify and fix issues early.

5.2 Integration Testing

Integration testing was used to verify that the modules work together as a complete system. For example, when a customer places an order, the cart, checkout, order, billing, and inventory modules all need to coordinate properly. Similarly, when a bill is generated, the sales table and email invoice workflow must receive the correct data.

This testing phase confirms that data passes correctly from one module to another and that database updates remain consistent. It also helps uncover issues caused by mismatched field names, missing session values, or incomplete form handling.

5.3 System Testing

System testing was performed to assess the application as a whole. The purpose was to evaluate whether the complete system supports the expected pharmacy workflow under realistic usage conditions. The test environment involved login attempts, record updates, order placement, checkout processing, and report viewing.

The focus of system testing was on functional completeness, navigation consistency, and correct interaction between pages. It also examined whether the interface remains usable when multiple roles access the application in sequence.

5.4 Validation Testing

Validation testing ensures that the application meets the original requirements. In this project, validation involved confirming whether the system actually provided multi-role login, inventory control, billing, order management, delivery tracking, and reporting as planned.

It was also necessary to verify that the application matched the practical expectations of a pharmacy workflow. Validation confirmed that the system is not only technically functional but also appropriate for the problem it was meant to solve.

5.5 Test Cases

Representative test cases for the system are shown below.

Table 5.1: Sample Test Cases

Test Case	Input Data	Expected Result	Actual Result
Valid login	Correct username and password	User redirected to dashboard	Passed
Invalid login	Wrong password	Error message displayed	Passed
Add medicine	Medicine details and quantity	Record saved in inventory	Passed

Test Case	Input Data	Expected Result	Actual Result
Create bill	Selected medicines and quantities	Invoice generated and stored	Passed
Low stock check	Quantity below threshold	Alert displayed to admin	Passed
Cart update	Increase or decrease quantity	Cart totals updated correctly	Passed
Order checkout	Confirmed cart items	Order recorded successfully	Passed
Delivery update	Status changed to delivered	Tracking record updated	Passed

5.6 Result Analysis

The testing results indicate that the system performs the core functions required for a smart pharmacy workflow. Login authentication works as intended, medicine and order records are handled accurately, and billing information is generated in a structured format. The inventory alerts support proactive stock management, while the tracking and reporting modules add visibility to operational data.

Although further hardening and feature expansion would be needed for large-scale deployment, the current version is suitable for academic demonstration and illustrates a coherent end-to-end workflow. Overall, the test outcomes show that the system is stable for the planned scope of the mini project.

Chapter 6

Results and Discussion

The implemented system provides a functional and organized interface for managing pharmacy activities. The role-based dashboards ensure that each user sees only the pages that are relevant to their responsibilities. The admin can manage inventory and reports, employees can support billing and delivery operations, and customers can place orders and review their order history.

The database-backed architecture improves reliability because every transaction is recorded and can be reviewed later. This is a major improvement over manual pharmacy records, where it is difficult to trace the history of a sale, identify stock changes, or confirm delivery progress. The automation of billing and invoice generation also reduces waiting time and improves service quality.

From an operational point of view, the most important benefit is visibility. The admin can monitor stock quantities, expiry status, and sales trends from the dashboard and reports. Customers can check their orders without calling the pharmacy repeatedly. Employees can use a smaller set of actions to complete routine tasks efficiently. These improvements collectively make the pharmacy more organized and responsive.

The placeholders below represent the principal screens of the application. They are included as figure environments so that the final report structure remains complete even before actual screenshots are inserted.

6.1 Screenshot Placeholders

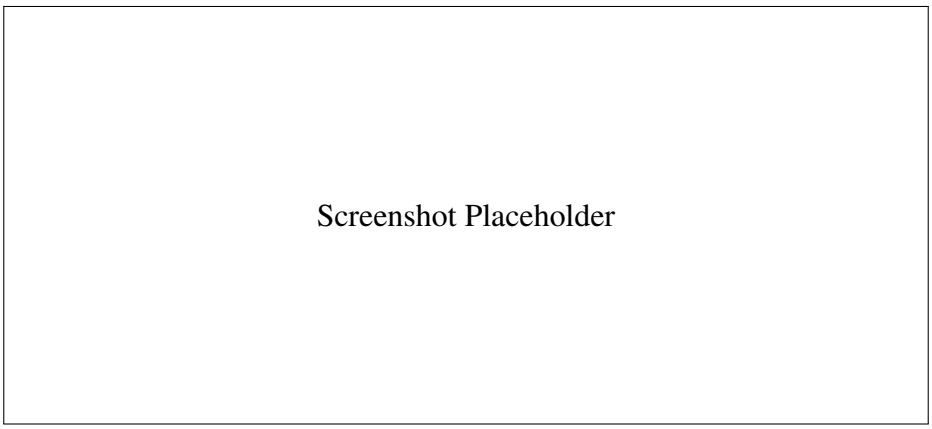


Figure 6.1: Login Page

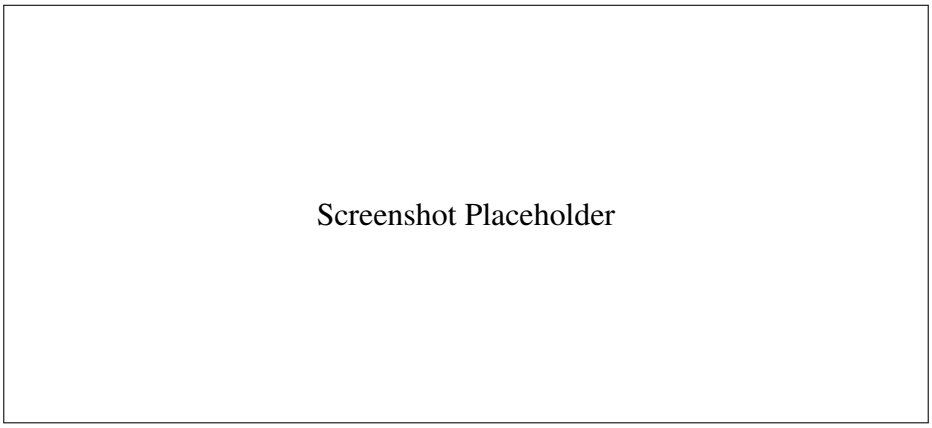


Figure 6.2: Registration Page

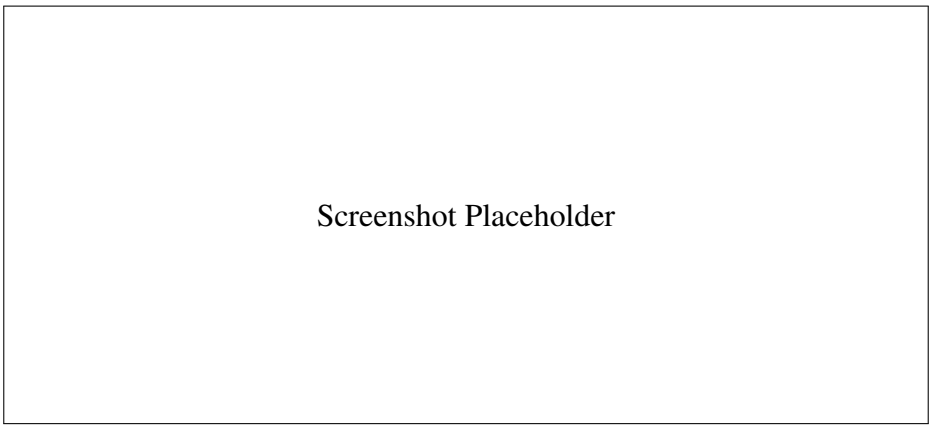


Figure 6.3: Admin Dashboard

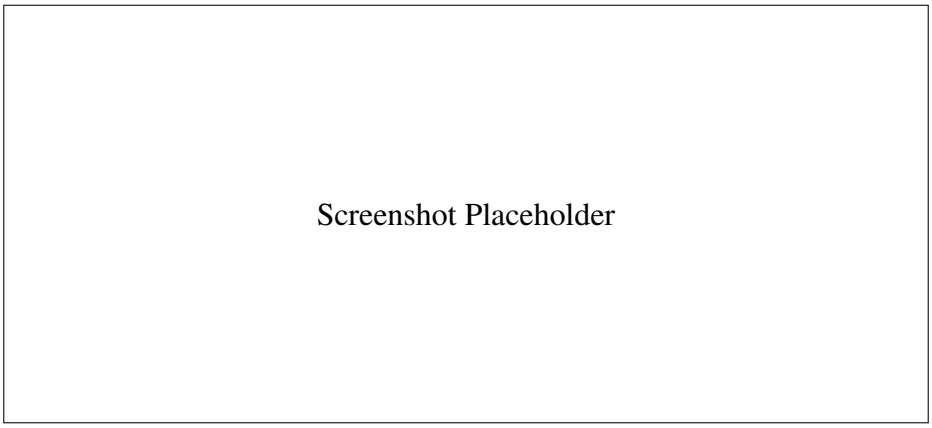


Figure 6.4: Billing Page

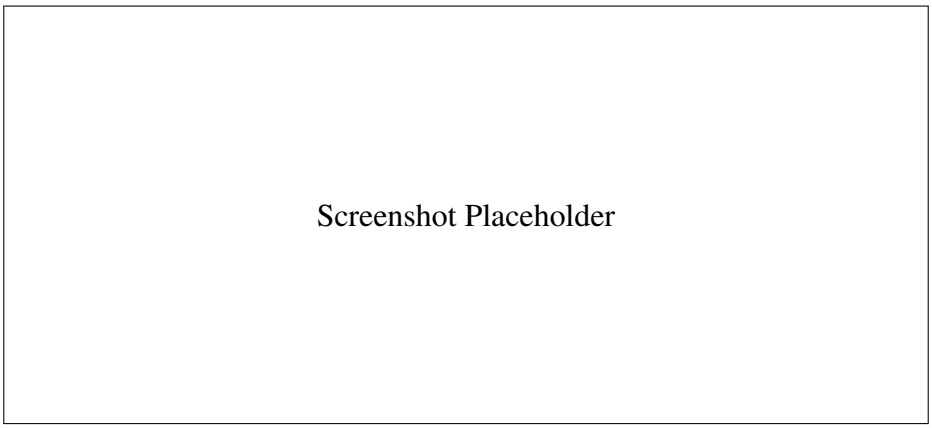


Figure 6.5: Invoice Generation

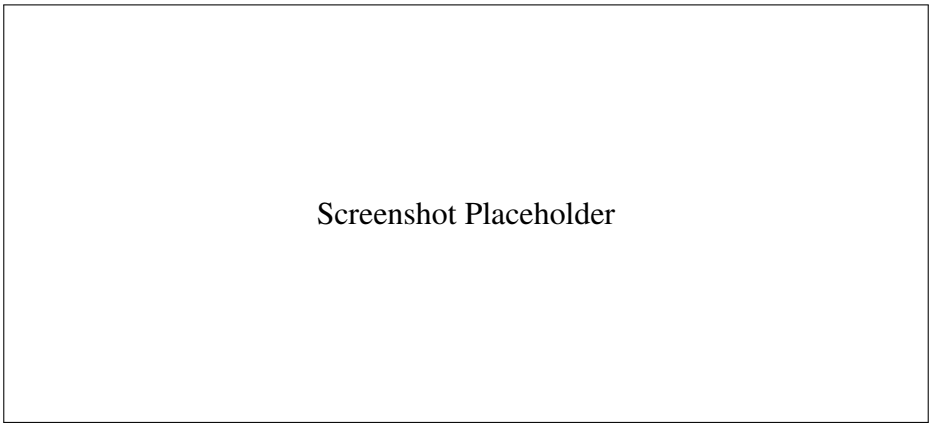


Figure 6.6: Medicines Page

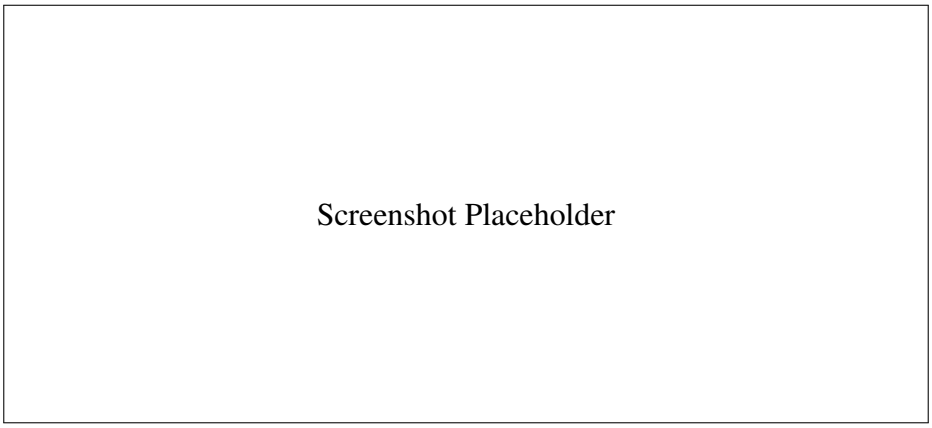


Figure 6.7: Reports Page

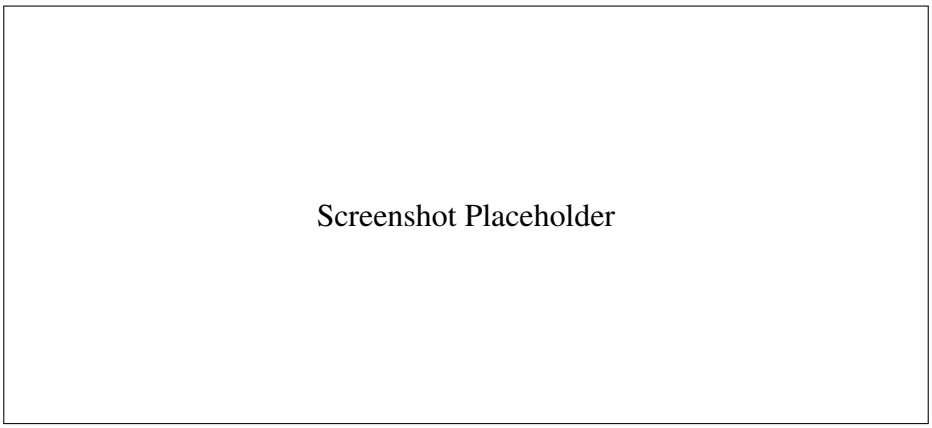


Figure 6.8: Delivery Tracking

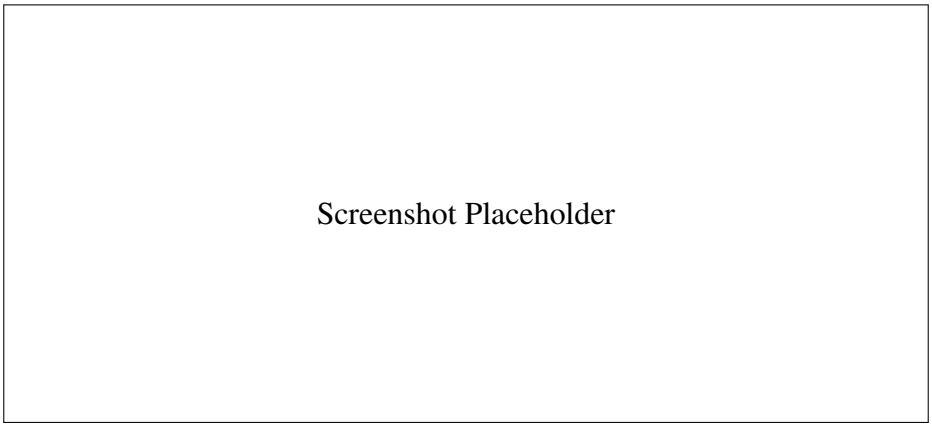


Figure 6.9: Customer Dashboard

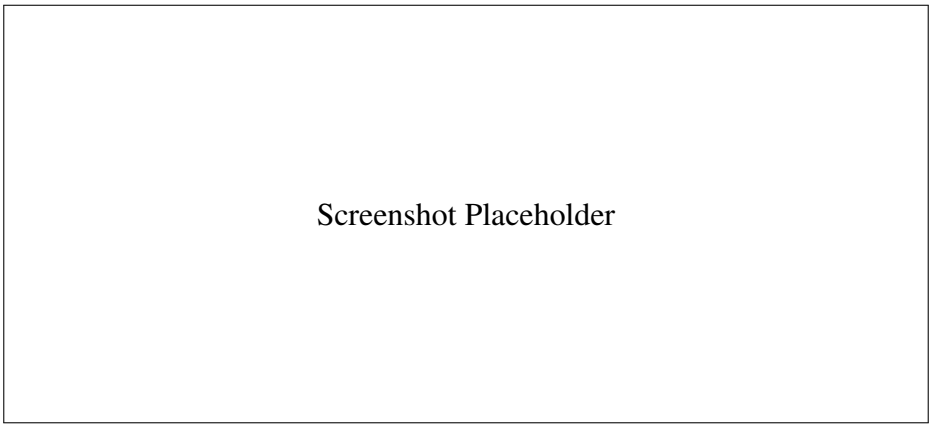


Figure 6.10: Shop Page

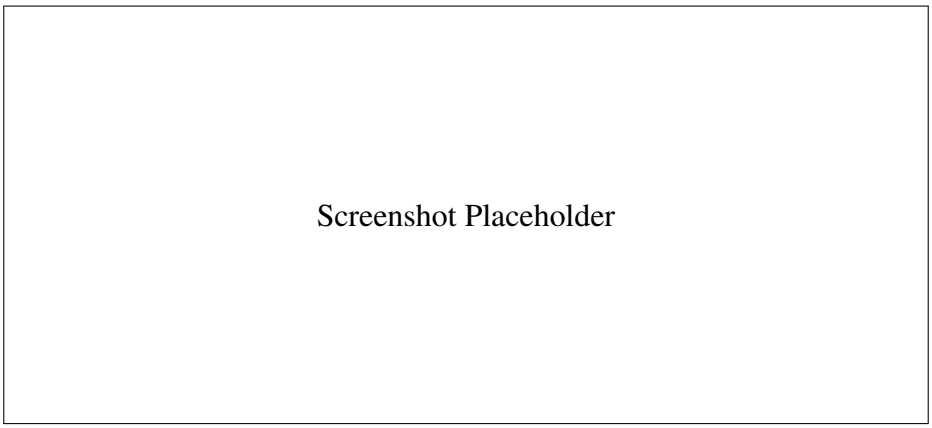


Figure 6.11: Cart Page

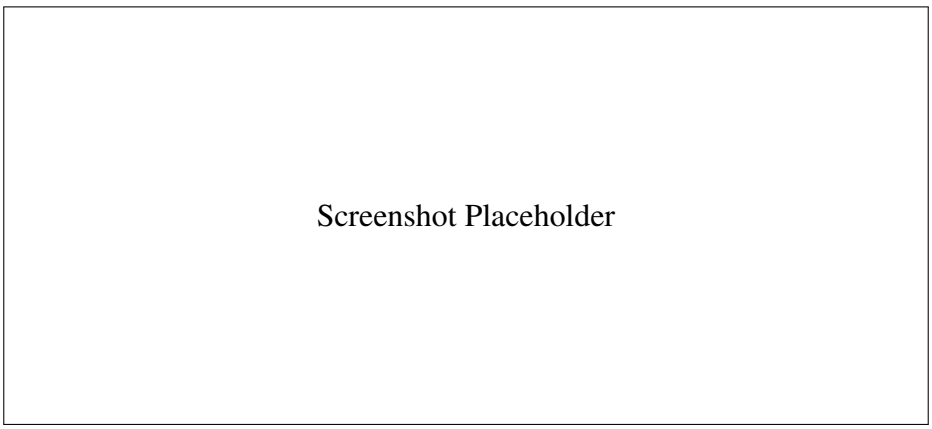


Figure 6.12: Checkout Page

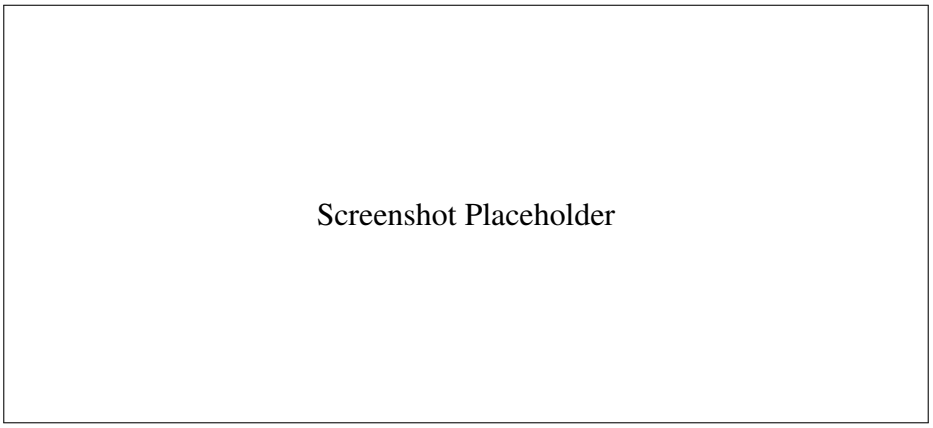


Figure 6.13: My Orders Page

Chapter 7

Conclusion and Future Enhancements

7.1 Conclusion

The Smart Pharmacy Inventory Management System demonstrates how a pharmacy can be managed more effectively through a structured web application. By combining inventory control, billing, order handling, customer services, delivery tracking, and report generation in one platform, the system addresses many of the issues found in manual or fragmented workflows.

The project confirms that a relatively compact technology stack such as PHP, MySQL, Bootstrap, and XAMPP is sufficient to implement a useful and practical business application. The role-based design improves access control and clarity, while the database-centered architecture provides reliable record keeping. The system therefore meets the key objectives of the mini project and serves as a strong academic demonstration of full-stack application development.

7.2 Future Enhancements

Several enhancements can be considered for future versions of the system:

- Integration with online payment gateways for fully digital checkout.
- Barcode or QR code support for faster medicine identification.
- SMS notifications for order status and delivery updates.
- Advanced analytics dashboard with charts and graphs.
- Mobile-responsive progressive web application support.

- Multi-branch pharmacy support with centralized stock synchronization.
- Role-based audit logs for stronger compliance and traceability.
- Automated purchase recommendation based on demand trends and stock turnover.

Future development can transform the current project into a more advanced enterprise solution. The modular structure of the design makes such expansion practical without rewriting the entire system.

References

Bibliography

[1] PHP Manual, official documentation for PHP language constructs and web development features.

[2] MySQL Reference Manual, official documentation for relational database management and SQL usage.

[3] Bootstrap Documentation, responsive frontend framework documentation and component reference.

[4] PHPMailer Documentation, email delivery library for PHP applications.

[5] Visvesvaraya Technological University project report guidelines and academic formatting references.